

Introduction to ROS1

M. J. Khojasteh

Fall 2026

1 Objective

This lab serves as an introduction to the fundamental **Command Line Interface** (CLI) tools within both the **Robotic Operating System** (ROS1). In this class we will use two versions of ROS, ROS1, and ROS2. The general development process between the two is very similar however this lab will only focus on ROS1. To use ROS1 you can use a docker dev container or VM on your personal computer, or you may use the lab computers. If you decide to use a virtual machine, you will need to create an account to download VMWare, but it shouldn't take too long. It is recommended that even if you decide to use the lab computers you should still install VMWare/Docker so that you can complete a few labs on your own time. This lab is designed to provide a foundation in understanding how to interface with ROS at a programmatic level using the terminal. This guide follows the documentation provided on the ROS website, there will be a hyperlink for each part to what tasks you should accomplish.

This lab does not repeat the content provided in this documentation; instead, the flow of the lab takes the following approach: You should run through each module of the Beginner tutorial in the documentation. Upon the completion of each module the lab will give pointers/best practices to follow before moving on to the next module.

Upon completion of this lab, students should be able to:

1. Identify and explore ROS1 packages and executables.
2. Execute ROS1 nodes from the terminal.
3. List, monitor (echo), and publish messages to ROS1 topics.

2 Introduction

2.1 Linux/Ubuntu

If you have never used Linux or Ubuntu before don't worry this should hopefully clear up some confusion, and as always your TA's are available if you want a deeper explanation. Linux is essentially an open-sourced operating system similar to Windows. Due to the open-sourced nature many companies turn to this operating system framework for development on applications, security, and privacy. Now, in Linux, there are what's called **Distributions**. You can think of distributions like "flavors". For example, let's say we have a piece of candy. This candy may have an assortment of flavors, but each piece of candy is made with the same ingredients. So under the hood each "flavor" of Linux runs off the same base-framework, Now just like candy, Linux has its own popular "flavors". In this case Ubuntu is one of the favored distributions due to its stability and widespread use. As such many companies make their applications under the assumption that you'll be using a popular distribution like Ubuntu. Each of these distributions also have version numbers, currently Ubuntu is in the stable version 24.04 (just means that this version has been profusely tested for bugs). Think of Ubuntu 24.04 like Windows 11. Now sometimes apps were created with the idea that you'd use a previous version of Ubuntu like version 22.04, think Windows 10. ROS which we will be using in these labs is one of those has different versions depending on the version of Ubuntu you use.

2.2 ROS

In this lab we will use ROS1, the version of ROS1 that we will be using is called "Noetic". This version only works on Ubuntu 20.04.

2.2.1 What is ROS?

ROS is an open-source, operating system for your robot. It provides services for a robot such as hardware abstraction, low-level device control, implementation of message passing, commonly used functions, and package management. ROS also provides tools and libraries for obtaining, building, writing, and running code across multiple computers.

ROS Publish/Subscribe Communication Model

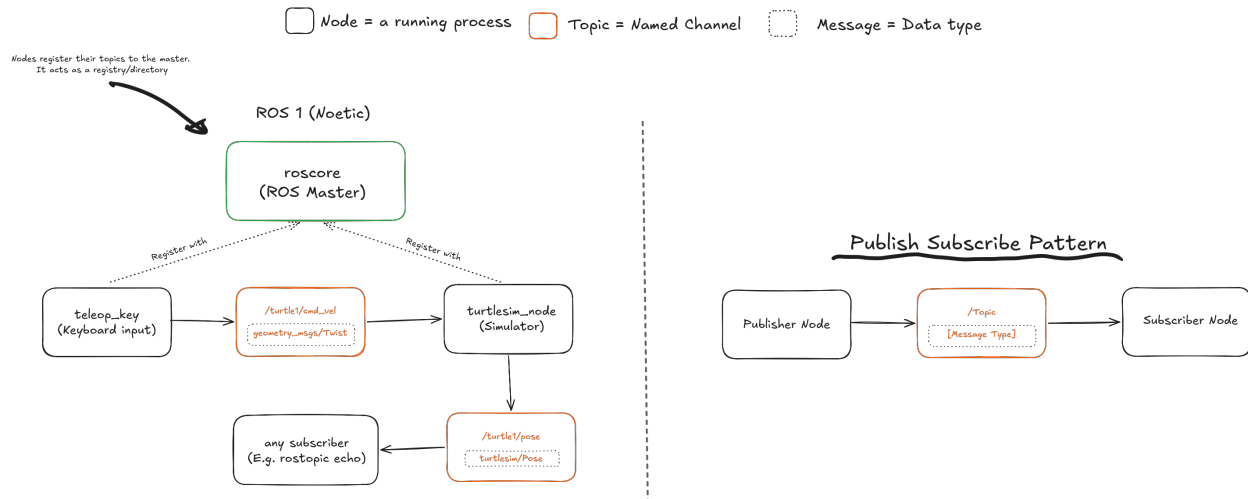


Figure 1: Overview of ROS Communication Structure

2.2.2 How and Why ROS is Used

The primary goal of ROS is to support code reuse in robotics research and development. ROS is a distributed framework of processes (aka Nodes) that enables executables to be individually designed and loosely coupled at runtime. These processes can be grouped into Packages and Stacks, which can be easily shared and distributed. ROS also supports a system of code Repositories that enable collaboration to be distributed as well. This design, from the file system level to the community level, enables independent decisions about development and implementation, but all can be brought together with ROS infrastructure tools. Packages are the main unit for organizing software in ROS. A package may contain ROS runtime processes (nodes), a ROS-dependent library, datasets, configuration files, or anything else that is usefully organized together. A single node can both publish and subscribe to ROS messages in much the same way that functions within simple programs can take in and return information.

3 ROS1 Tutorials

3.1 Beginner

3.1.1 Installing and Configuring ROS Environment

1. Please follow the tutorial for Installing and Configuring ROS Environment
2. The most important thing to note from this is that you **MUST** either source your workspace for every terminal/tab you open, or add the command to your shell startup script.
3. When we source our workspace, we are telling ROS where to find relevant packages and/or executables.

3.1.2 Navigating the ROS Filesystem

1. Please follow the tutorial for Navigating the ROS Filesystem

3.1.3 Creating a ROS Package

1. Please follow the tutorial for Creating a ROS Package
2. The only difference for between typical Linux commands and the ROS version of them, is that the ROS version knows where packages are installed, and is "smarter" when it comes to the functionality of ROS.
3. For our purposes you can still use basic Linux commands, or you can choose to use the ROS ones, it's personal preference.

3.1.4 Building a ROS Package

1. Please follow the tutorial for Building a ROS Package
2. Understanding what makes a package is crucial, however don't worry too much about this, typically we won't be creating our own packages. Rather what you'll find is that you will be importing other packages.

3.1.5 Understanding ROS Nodes

1. Please follow the tutorial for Understanding ROS Nodes
2. When working with ROS1, make sure to have one of your terminal tabs always running `roscore` if you plan to run nodes, or topics.
3. Nodes are integral to how we use ROS, make sure you understand what they do.

3.1.6 Understanding ROS Topics

1. Please follow the tutorial for Understanding ROS Topics
2. `rqt_graph` is extremely handy and should be used often when you are confused about how nodes interact with each other. When in doubt fall back on this.
3. Topics are the gateway of communication between nodes.
4. Pay attention to how we use `rosmmsg`, and `rostopic`, this will be important when we implement our own publisher and subscriber. Communication between nodes requires the data structures to be equal.

3.1.7 Understanding ROS Services and Parameters

1. Please follow the tutorial for Understanding ROS Services and Parameters
2. Don't worry about this as much, services and parameters won't be heavily focused on during these labs. Just know that you CAN use them as a tool in your pocket.

3.1.8 Using `rqt_console` and `roslaunch`

1. Please follow the tutorial for Using `rqt_console` and `roslaunch`
2. `rqt_console` is handy when you need to view what went wrong with something in your tangled mess of nodes and topics (if you ever get to that point).
3. Using `roslaunch` files allows us to launch multiple nodes at once, instead of running each executable individually.

3.1.9 Using `roscd` to Edit Files in ROS

1. Please follow the tutorial for Using `roscd` to Edit Files in ROS
2. If you're a terminal wizard, by all means feel free to use this. You can also navigate within the folder system and open files with VSCode.

3.1.10 Creating a ROS msg and srv

1. Please follow the tutorial for Creating a ROS msg and srv

3.1.11 Write a Simple Publisher Subscriber

1. Please follow the tutorial for Write a Simple Publisher Subscriber
2. We chose python because it's a scripting language, but if you are more comfortable with C++, feel free to use it.

3.1.12 Examining the Simple Publisher Subscriber

1. Please follow the tutorial for Examining the Simple Publisher Subscriber
2. Make sure you understand how to use publishers and subscribers, they are pretty much the foundation of ROS. Make sure to ALWAYS check that your data structures being passed are the same.

4 Deliverables

1. Create a simple publisher and subscriber loop that publishes increasing numbers in either python or C++.
2. Create a custom message.
3. Create a publisher and subscriber node pair that uses your custom message.
4. Produce an image of the node network using `rqt_graph`.
5. Make sure to answer how a publisher subscriber network works and what the `rqt_graph` means.

5 Report

Make sure to submit a tech memo which follows the sekeleton/guidelines that are located in MyCourses.

You only need to submit ONE report for both part 1 and part 2 of this lab. Your report should include the following:

1. All deliverables.
2. Answers to all questions asked within the lab document.
3. A summary of what was accomplished and learned within the lab.